МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет
«Харківський авіаційний інститут»

Факультет систем управління літальних апаратів
Кафедра систем управління літальних апаратів

Лабораторна робота № 6

з дисципліни «Алгоритмізація та програмування»
на тему «Реалізація алгоритмів обробки одновимірних масивів мовою С ++»

ХАІ.301. группа 319а 18 ЛР

Виконав студент гр. _____319а_____

_____*Тучак Владислав*

*Олександрович*_____

(підпис, дата) (П.І.Б.)

Перевірив

_____ к.т.н., доц. Олена ГАВРИЛЕНКО

(підпис, дата) (П.І.Б.)

2026

МЕТА РОБОТИ

Вивчити теоретичний матеріал з основ представлення одновимірних і масивів на мові С ++ і реалізувати декларацію, введення з консолі, обробку і виведення в консоль одновимірних масивів на мові С ++ в середовищі QtCreator.

ПОСТАНОВКА ЗАДАЧІ

Завдання 1. Вирішити завдання на аналіз і виведення елементів одновимірного масиву. Розмір масиву і його елементи ввести з консолі. Спершу вивести весь масив у рядок в порядку зростання індексів, потім – елементи чи підраховані результати відповідно до завдання.

Завдання 2. Вирішити завдання на перетворення одновимірного масиву. Розмір масиву і його елементи ввести з консолі. Спершу вивести у консоль заданий масив, потім – змінений.

Завдання 3. У функції main() організувати багаторазовий вибір одного з двох завдань. Кожне завдання описати окремою функцією без параметрів. Введення, виведення, обробку масивів реалізувати окремими функціями з параметрами.

Залишити назву і номер, наприклад: Integer 25, табл.2 – 15, варіант 5

ВИКОНАННЯ РОБОТИ

Завдання 1 (Array49)

Умова: Дано цілочисельний масив розміру N. Якщо він є перестановкою чисел від 1 до N, вивести 0; інакше вивести номер першого неприпустимого елемента.

Вхідні дані:
N — розмір масиву, int, N > 0.
A[i] — елементи масиву, int, i = 1..N.

Вихідні дані:
k — результат перевірки:

0, якщо масив є перестановкою 1..N;
i, якщо A[i] не входить у діапазон 1..N або порушена унікальність (перший дубль).

Основна ідея:

1. Перевірити кожен елемент на належність діапазону [1; N].

2. Перевірити відсутність повторів за допомогою допоміжного масиву позначок used.

3. Якщо порушень немає — вивести 0.

Завдання 2 (Array102)

Умова: Дано масив розміру N і число K ($1 \le K \le N$). Після елемента з номером K вставити новий елемент зі значенням 0.

Вхідні дані:
N — розмір масиву, int, $N > 0$.
K — позиція вставки, int, $1 \le K \le N$.
A[i] — елементи масиву, int, i = 1..N.

Вихідні дані:
Масив розміру N+1, у якому після A[K] вставлено 0.

Основна ідея:

1. Збільшити розмір масиву на 1.

2. Зсунути елементи праворуч від кінця до позиції K+1.

3. Присвоїти A[K+1] = 0.

4. Вивести оновлений масив.

Завдання 3 (Меню 1/2, багаторазово)

Умова: Реалізувати меню для багаторазового запуску задач 1 і 2.

Пункти меню:
1 — виконати Array49.
2 — виконати Array102.
0 — вихід.

Основна ідея:

1. У циклі показувати меню і зчитувати вибір користувача.

2. Через switch виконувати відповідну задачу.

3. При виборі 0 завершувати програму.

4. При некоректному пункті виводити повідомлення про помилку.

ВИСНОВКИ

Було вивчено способи обробки одномірних масивів: перевірку на перестановку чисел від 1 до N та вставку нового елемента в задану позицію. Закріплено на практиці роботу з циклами, умовними операторами й меню багаторазового вибору, що відпрацьовано в коді програми. Отримано навички аналізу коректності вхідних даних і покрокової перевірки результатів виконання алгоритмів

ДОДАТОК А
Лістинг коду програми

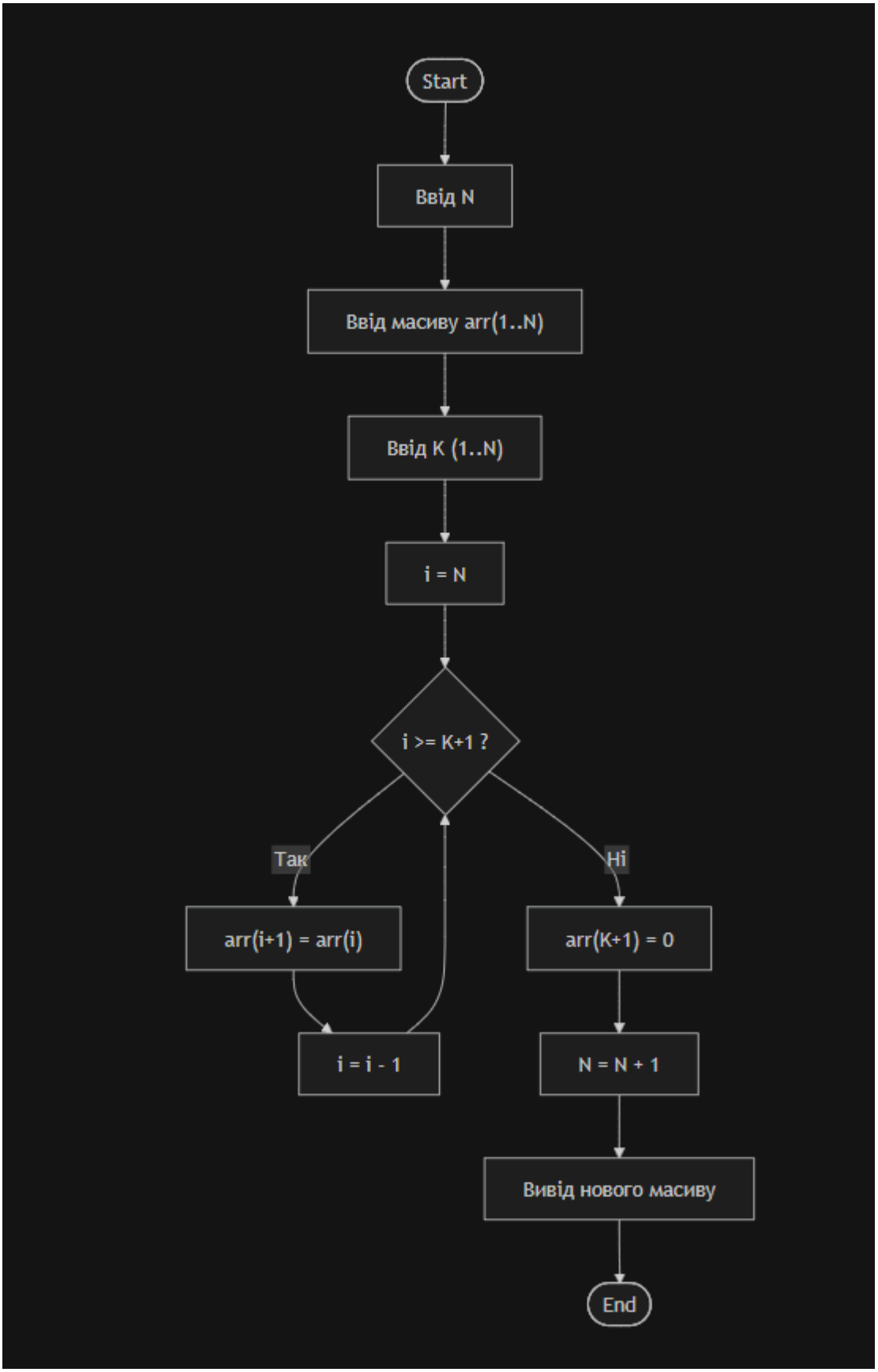
```
 #include <iostream>
#include <limits>
#include <string>

using namespace std;

const int N = 50; // максимальний розмір ВХІДНОГО масиву

// ---------- Допоміжні функції введення ----------
int input_int(const string& prompt) {
    int value;
    while (true) {
        cout << prompt;
        cin >> value;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Error! Please enter an integer value.\n";
        } else {
            return value;
        }
    }
}

int input_int_range(const string& prompt, int minVal, int maxVal) {
    int value;
    while (true) {
        value = input_int(prompt);
        if (value < minVal || value > maxVal) {
             cout << "Error! Please enter a value in range [" << minVal << ".."
<< maxVal << "].\n";
        } else {
            return value;
        }
    }
}

// ---------- Ввід/вивід масиву ----------
void input_array(int arr[], int &n) {
    n = input_int_range("Enter number of elements n (2..50): ", 2, N);
    for (int i = 0; i < n; i++) {
        arr[i] = input_int("A[" + to_string(i + 1) + "] = ");
    }
}
```

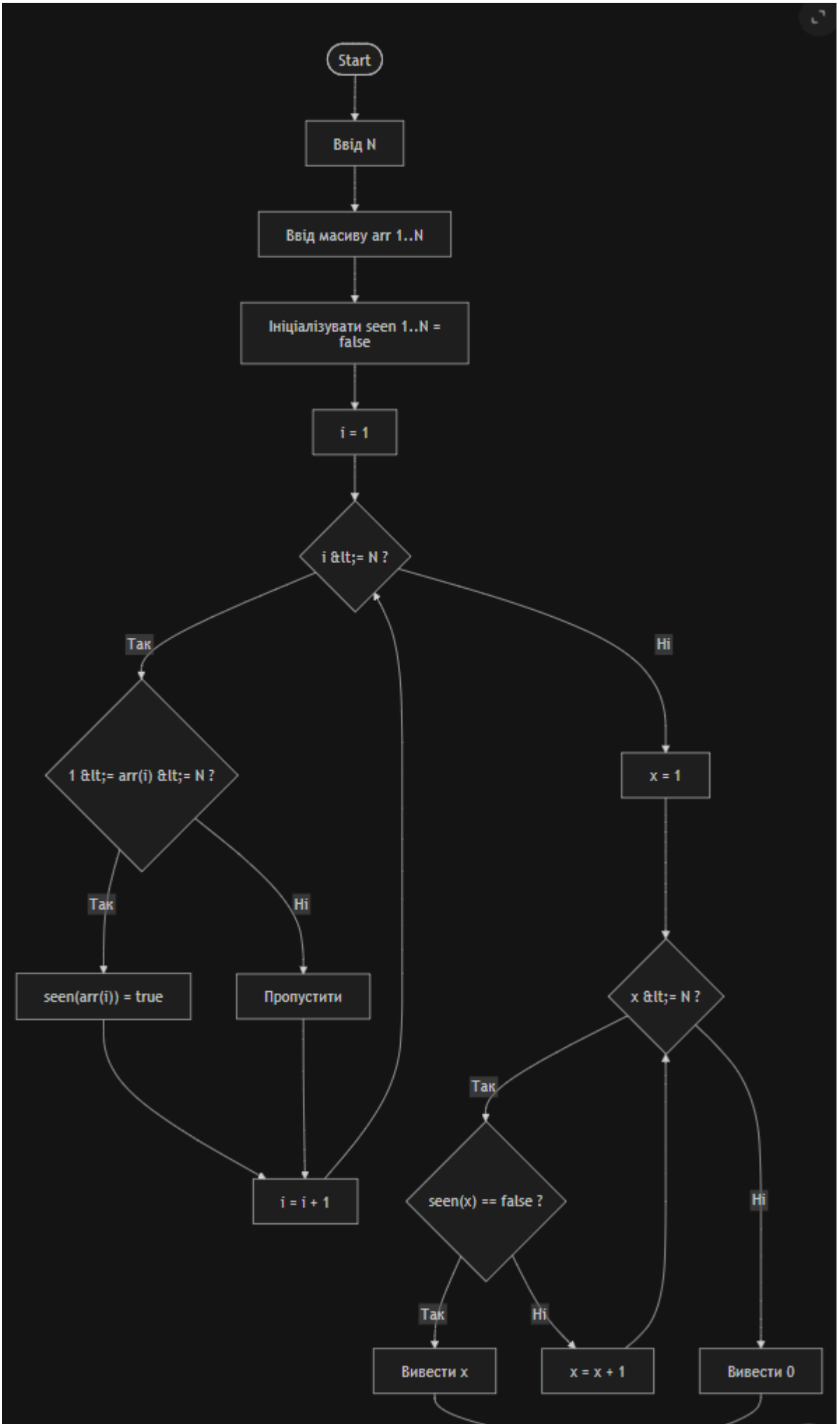
```
void output_array(const int arr[], int n) {
    cout << "Array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << (i + 1 == n ? "" : " ");
    }
    cout << "\n";
}

// ========================================================
// Завдання 1 (Таблиця 1) — Array49
// Дан цілочисельний масив розміру N. Якщо він є перестановкою (1..N),
// то вивести 0; інакше — номер першого неприпустимого елемента.
// ========================================================
int first_invalid_for_permutation_1_to_n(const int arr[], int n) {
    // used[v] = чи вже зустрічали число v
    bool used[N + 1] = {false};

    for (int i = 0; i < n; i++) {
        int x = arr[i];

        // 1) поза діапазоном 1..n
        if (x < 1 || x > n) {
            return i + 1; // 1-based позиція
        }

        // 2) дубль
        if (used[x]) {
            return i + 1; // перша позиція, де стало "не можна"
        }

        used[x] = true;
    }

    // якщо всі числа 1..n зустрілись рівно по 1 разу
    return 0;
}

void task_array49() {
    cout << "\n=== Task Array49 (Table 1) ===\n";
    int arr[N];
    int n;

    input_array(arr, n);
    output_array(arr, n);

    int res = first_invalid_for_permutation_1_to_n(arr, n);
      cout << "Answer (0 if it is a permutation, otherwise position of first
invalid element): "
         << res << "\n";
```

```
}

// ========================================================
// Завдання 2 (Таблиця 2) — Array102
// Дан масив розміру N і ціле число K (1 ≤ K ≤ N).
// ПІСЛЯ елемента з номером K вставити новий елемент зі значенням 0.
// Без допоміжних масивів.
// ========================================================
void insert_zero_after_k(int arr[], int &n, int k) {
    // k — 1-based (1..n)
    // вставка ПІСЛЯ k-го => новий елемент стане на позицію k+1 (1-based),
    // тобто індекс k (0-based)
    int pos = k; // 0-based індекс вставки

    // Зсуваємо вправо хвіст масиву на 1, починаючи з кінця
    for (int i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }

    arr[pos] = 0;
    n++;
}

void task_array102() {
    cout << "\n=== Task Array102 (Table 2) ===\n";
    int arr[N + 1]; // запас 1 елемент для вставки нуля
    int n;

    input_array(arr, n);

    int k = input_int_range("Enter K (1..n): ", 1, n);

    cout << "\nSource array:\n";
    output_array(arr, n);

    insert_zero_after_k(arr, n, k);

    cout << "Modified array (0 inserted after element #" << k << "):\n";
    output_array(arr, n);
}

// ========================================================
// Завдання 3 — меню вибору 1/2, багаторазово
// ========================================================
int main() {
    cout << "Lab 6. One-dimensional arrays (Array49 + Array102)\n";

    while (true) {
        cout << "\nMenu:\n";
        cout << "1 - Task 1 (Array49: permutation check)\n";
```

```
        cout << "2 - Task 2 (Array102: insert 0 after K-th element)\n";
        cout << "0 - Exit\n";

        int choice = input_int_range("Your choice: ", 0, 2);

        if (choice == 0) {
            cout << "Program finished.\n";
            break;
        } else if (choice == 1) {
            task_array49();
        } else { // choice == 2
            task_array102();
        }
    }

    return 0;
}
```

Діаграми


Start
Ввід N
Ввід масиву arr(1..N)
Ввід K (1..N)
i = N
i >= K+1 ?
Так
Ні
arr(i+1) = arr(i)
arr(K+1) = 0
i = i - 1
N = N + 1
Вивід нового масиву
End


Start
Ввід N
Ввід масиву arr 1..N
Ініціалізувати seen 1..N = false
i = 1
i <= N ?
Так
Ні
1 <= arr(i) <= N ?
Так
Ні
seen(arr(i)) = true
Пропустити
i = i + 1
x = 1
x <= N ?
Так
Ні
seen(x) == false ?
Так
Ні
Вивести x
x = x + 1
Вивести 0

ДОДАТОК Б

options | compilation | execution

```
Lab 6. One-dimensional arrays (Array49 + Array102)

Menu:
1 - Task 1 (Array49: permutation check)
2 - Task 2 (Array102: insert 0 after K-th element)
0 - Exit
Your choice:
```

options | compilation | execution

```
Lab 6. One-dimensional arrays (Array49 + Array102)

Menu:
1 - Task 1 (Array49: permutation check)
2 - Task 2 (Array102: insert 0 after K-th element)
0 - Exit
Your choice: 1

=== Task Array49 (Table 1) ===
Enter number of elements n (2..50): 2
A[1] = 4
A[2] = 6
Array: 4 6
Answer (0 if it is a permutation, otherwise position of first invalid element): 1

Menu:
1 - Task 1 (Array49: permutation check)
2 - Task 2 (Array102: insert 0 after K-th element)
0 - Exit
Your choice:
```

```
=== Task Array102 (Table 2) ===
Enter number of elements n (2..50): 5
A[1] = 5
A[2] = 3
A[3] = 5
A[4] = 5
A[5] = 61
Enter K (1..n): 1

Source array:
Array: 5 3 5 5 61
Modified array (0 inserted after element #1):
Array: 5 0 3 5 5 61

Menu:
1 - Task 1 (Array49: permutation check)
2 - Task 2 (Array102: insert 0 after K-th element)
0 - Exit
Your choice:
```

ДОДАТОК В

Діалог з ШІ для самоаналізу

1) Промпт студента

Ти - викладач, що приймає захист моєї роботи. Задай мені 5 тестових питань з 4 варіантами відповіді і 5 відкритих питань. Це мають бути завдання середнього рівня складності на розвиток критичного та інженерного мислення. Питання мають відноситись до коду лабораторної роботи (Array49, Array102, меню 1/2) і до теоретичних відомостей про масиви, алгоритми та перевірку коректності даних.

---

1) Відповідь ШІ (питання)

Тестові питання (1 правильна відповідь):

1. У задачі Array49 масив є коректною перестановкою 1..N, якщо:
A) всі елементи парні
B) кожен елемент належить [1..N] і не повторюється
C) масив відсортований за зростанням
D) сума елементів дорівнює N
Правильна відповідь: B

2. Яка структура найзручніша для перевірки повторів у Array49 за O(N)?
A) вкладені цикли
B) допоміжний масив позначок used
C) сортування + лінійний пошук
D) рекурсія без додаткової пам'яті
Правильна відповідь: B

3. У задачі Array102 після вставки 0 після K-го елемента потрібно:
A) видалити останній елемент
B) зсунути елементи праворуч від кінця до позиції K+1
C) відсортувати масив
D) замінити всі елементи після K на 0
Правильна відповідь: B

4. Що є критичним для коректності меню (1/2/0)?
A) використання тільки for-циклу
B) багаторазове повторення через цикл і обробка коректного вводу
C) вивід без перевірок

4. Що є критичним для коректності меню (1/2/0)?

A) використання тільки for-циклу

B) багаторазове повторення через цикл і обробка некоректного вводу

C) вивід без перевірок

D) відсутність пункту виходу

Правильна відповідь: B

5. Яка асимптотична складність перевірки Array49 з used?

A) O(1)

B) O(log N)

C) O(N)

D) $O(N^2)$

Правильна відповідь: C

Відкриті питання:

6. Поясни, чому перевірка "тільки діапазону [1..N]" недостатня для Array49.

7. Опиши, як обробляються граничні випадки в Array102 (K=1, K=N, N=1).

8. Які помилки користувацького вводу слід передбачити в меню програми?

9. Порівняй два підходи перевірки перестановки: через used і через сортування.

10. Запропонуй мінімум 5 тест-кейсів для перевірки всіх трьох частин роботи.

---

2) Промпт студента

**Мої відповіді:

1-B, 2-B, 3-B, 4-B, 5-C.

6) Бо елементи можуть повторюватися.

7) K=1 — вставка після першого; K=N — у кінець; N=1 — масив стане з 2 елементів.

8) Неправильний пункт меню, нечисловий ввід, пустий ввід.

9) used швидше O(N), сортування простіше в реалізації, але повільніше O(N log N).

10) Тести:

- Array49: [1,2,3,4] -> 0
- Array49: [1,2,2,4] -> помилка
- Array49: [0,2,3,4] -> помилка
- Array102: N=5,K=3,[1,2,3,4,5] -> [1,2,3,0,4,5]
- Меню: вибір 1,2,0 і некоректний 9.

2) Відповідь ШІ (аналіз і оцінки)

Нижче оцінка по кожному пункту (5-бальна шкала):

1. **5/5** — правильна відповідь. Ймовірність ШІ: низька.
2. **5/5** — правильна відповідь. Ймовірність ШІ: низька.
3. **5/5** — правильна відповідь. Ймовірність ШІ: низька.
4. **5/5** — правильна відповідь. Ймовірність ШІ: низька.
5. **5/5** — правильна відповідь. Ймовірність ШІ: низька.
6. **4.5/5** — суть правильна, але варто додати приклад (напр. [1,1,3,4]). Ймовірність ШІ: середня.
7. **4.5/5** — коректно, бажано додати опис зсуву елементів. Ймовірність ШІ: середня.
8. **4/5** — добре, але бракує обробки виходу за межі діапазону та повторного запиту при помилці. Ймовірність ШІ: середня.
9. **5/5** — сильне порівняння складності та практичності. Ймовірність ШІ: середня.
10. **4.5/5** — хороші тести, бажано додати крайній випадок Array102: K=N. Ймовірність ШІ: середня.

Штраф -60% не застосовано, оскільки високої ймовірності використання ШІ саме у відповідях не виявлено (відповіді короткі, предметні, з ознаками власного розуміння).

Середній бал:

$$(5+5+5+5+5+4.5+4.5+4+5+4.5)/10 = 4.75$$

Підсумок: 4.75 / 5.00

---

3) Промпт студента

Проаналізуй код у звіті (Array49, Array102, меню), і додай опис і приклади коду з питань, які є в теоретичних відомостях, але не відпрацьовано у коді при вирішенні завдань.

---

3) Відповідь ШІ (що додати до коду)

За змістом роботи практично покрито базові операції з масивами та керуючими конструкціями. Теми, які часто є в теорії, але частково не відпрацьовані:

1. **Валідація вводу через** `cin.fail()`